

Fingerprint Re-Identification (SOCOFing)

An open-set fingerprint identification/verification system trained on [SOCOFing](#). Instead of a fixed-class classifier, the model learns an **embedding space**: a CNN maps a fingerprint image to a 128-D vector such that images of the same finger land close together and images of different fingers land far apart, including fingers never seen during training. That is what makes it usable for real-world enrollment, a new person can be added to the gallery without retraining the network.

Trained with batch-hard triplet loss (Hermans et al., 2017) and a strict subject-level train/val/test split, so evaluation reflects genuinely unseen identities.

Results

Evaluated on the held-out test split (90 subjects / ~900 fingers never seen during training).

Gallery = clean Real images (one per finger),

Probe = Altered images (synthetic distortions simulating degraded scans).

Alteration level	Rank-1	Rank-5	EER	Mean genuine dist.	Mean impostor dist.
Easy	100%	100%	0.0067%	0.0538	1.3725
Medium	100%	100%	0.0330%	0.0798	1.3705
Hard	100%	100%	0.0023%	0.1036	1.3707
Pooled (all)	100%	100%	0.0135%	0.0773	1.3723

The smooth, monotonic increase in genuine-pair distance with alteration severity (0.054 $\rightarrow$ 0.080 $\rightarrow$ 0.104), while impostor distance stays flat (~1.37), is the signature of genuine ridge structure matching rather than the model exploiting alteration-type artifacts as a shortcut — see “Notes on evaluation” below for why that distinction mattered here.

Reproduce with:

```
python eval.py --data_root /path/to/SOCOFing --checkpoint ./checkpoints/best_model.pt
```

Architecture

model.py : a small ResNet-style CNN sized for 96 x 96 grayscale fingerprint crops:

- Strided convolutions instead of max-pooling for downsampling, to preserve ridge-orientation detail.
- Global average pooling before the embedding head, for robustness to small translation/rotation offsets between scans.
- L2-normalized 128-D output (embeddings live on the unit hypersphere, so distance behaves like cosine similarity).

losses.py : batch-hard triplet loss :- for every anchor in a P-K batch (P distinct identities $\times$ K images each), mines the hardest same-identity positive and hardest different-identity negative actually present in the batch, rather than relying on random triplet sampling (most random triplets are already “easy” and contribute ~zero gradient).

A note on the debugging history

This model went through a real embedding-collapse bug during development that is worth documenting so it is not reintroduced:

The bug: SOCOFing filenames encode a person id, but each person contributes 10 different fingers.. Earlier versions of the codebase used that person ID directly as the triplet-loss identity label. That meant batch-hard mining would regularly treat two structurally unrelated fingers of the same person (e.g. their thumb and pinky) as a “positive pair” the network should make similar, actively fighting the network's ability to learn discriminative ridge structure. The result was full embedding collapse (all images mapping to nearly the same point), which optimizer-side fixes (learning rate, weight decay and distance-metric numerics) could not resolve, because the objective itself was fighting the data.

The fix: `SocofingRecord.finger_uid (subject_id + hand + finger)` is now the identity used for triplet mining (dataset.py's label maps key on this). The train/val/test split still holds out whole people, not individual fingers

(`socofing_index.py's split_subjects_train_val_test groups by subject_id`) this is intentional: a person's fingers can share correlated skin/texture characteristics, so holding out only one of their ten fingers could still leak person-specific information across splits.

If you are extending this codebase: triplet-mining labels should always key on `finger_uid` and dataset splits should always key on `subject_id`. Mixing these up is exactly what caused the original collapse.

Notes on evaluation

Rank-k identification accuracy and EER (Equal Error Rate) were used rather than classification accuracy because this is an open-set embedding model — it must work on identities never seen during training, which a fixed softmax classifier cannot do. These are the standard metrics in face/fingerprint/speaker verification literature:

- Rank-1 / rank-5 accuracy: for a 1:N identification task (given a probe, find who it belongs to among everyone enrolled), is the correct match the top result or in the top 5?
- EER: for a 1:1 verification task (is this probe the same finger as this specific enrolled entry?), the threshold where false-accept rate equals false-reject rate.

Training-time (train_loss/val_loss/active_triplets) are useful for monitoring training health (they are what caught the collapse bug above) but do not by themselves confirm real-world matching performance. A triplet loss near zero just means sampled triplets satisfy the margin, which gets trivially easy once classes separate at all. The gallery/probe evaluation above is the metric that actually matters.

Setup

```
python -m venv .venvsource .venv/bin/activate # or .venv\Scripts\activate on Windows  
pip install torch numpy pillow
```

Download [SOCOFing](#) and note the path to the folder that directly contains `Real/` and `Altered/`.

Training

```
python train.py --data_root /path/to/SOCOFing \ --epochs 30 --p 16 --k 4 --lr 1e-4 --warmup_steps 500 \  
--out_dir ./checkpoints
```

Key hyperparameters

Flag	Default	Notes
--embedding_dim	128	
--margin	0.3	Triplet loss margin
--p / --k	16 / 4	Distinct identities / images-per-identity per batch
--lr	1e-4	Peak LR after warmup
--warmup_steps	500	Linear warmup before cosine decay
--weight_decay	1e-4	Applied via AdamW; excluded for BatchNorm/bias params
--grad_clip	1.0	Max gradient norm
--collapse_std_threshold	0.02	Warns if embedding pairwise-distance std drops below this

The training loop prints an embedding-collapse warning every epoch if the pairwise-distance std drops too low — if this warning persists for many epochs, check the identity-labeling note above before assuming it is a learning-rate issue.

Checkpoints (`best_model.pt`, `final_model.pt`) are saved to `--out_dir`. Each run starts training from a fresh random initialization; there is no resume-from-checkpoint flag.

Evaluation

```
# Full test set, all alteration levels
python eval.py --data_root /path/to/SOCOFing --checkpoint
./checkpoints/best_model.pt
```

```
# Per-alteration-level breakdown:
```

```
python eval.py --data_root /path/to/SOCOFing --checkpoint ./checkpoints/best_model.pt --include_altered_levels
Easy
```

```
python eval.py --data_root /path/to/SOCOFing --checkpoint ./checkpoints/best_model.pt --include_altered_levels
Medium
```

```
python eval.py --data_root /path/to/SOCOFing --checkpoint ./checkpoints/best_model.pt --include_altered_levels
Hard
```

--seed must match the value used for training (default 42 in both scripts) so the test split reproduces the exact same held-out subjects the model never saw.

License

MIT License

Copyright (c) 2026 Naman Bhatia

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.